

Practical - 1

Aim: Introduction and Installation of Node.js.

Discuss the following points:

Q1: How JavaScript Behaves differently in browser and in a Node.js environment?

Ans:

Node.js and browsers both run JavaScript, but they have different environments and capabilities.

Node.js is designed for server-side development, while browsers are for client-side applications.

- **APIs:** Node.js provides APIs for file system, networking, and OS, which browsers do not. Browsers provide DOM, fetch, and UI APIs not available in Node.js.
- **Global Objects:** Node.js uses global; browsers use window or self.
- **Modules:** Node.js uses CommonJS (require) and ES modules (import); browsers use ES modules or plain `<script>` tags.
- **Security:** Browsers run in a sandbox with limited access; Node.js has full access to the file system and network.
- **Event Loop:** Both environments use an event loop, but Node.js has additional APIs for timers, process, etc.
- **Environment Variables:** Node.js can access environment variables (process.env); browsers cannot.
- **Package Management:** Node.js uses npm/yarn; browsers use CDNs or bundlers.

Feature	Node.js	Browser
File System Access	Yes	No
Networking (TCP/UDP)	Yes	No
DOM Access	No	Yes
Global Object	global	window/self
Modules	CommonJS/ESM	ESM/Scripts
Environment Variables	Yes (process.env)	No
Security	Full OS access	Sandboxed
Package Management	npm/yarn	CDN/Bundler

Q2: Roles of Front-end and Back-end in Full-Stack Development and Node.js's Role.

Ans:

Front-end

- User interface of a website/application.
- Built using HTML, CSS, and JavaScript.
- Runs in the browser.

Back-end

- Handles business logic, databases, authentication, and APIs.
- Processes requests and sends responses.

Role of Node.js

- Node.js acts as the backend runtime environment.
- Allows developers to use JavaScript on the server side.
- Helps build APIs, web servers, real-time applications, and microservices.

Q3: Define Node.js. What are its key features and primary purpose?

Ans:

Node.js is an open-source, cross-platform JavaScript runtime environment built on Google's V8 JavaScript engine that executes JavaScript code outside the browser.

Key Features

- Asynchronous and event-driven architecture
- Non-blocking I/O operations
- Fast execution using V8 engine
- Cross-platform support
- Large package ecosystem through NPM
- Scalable and lightweight

Primary Purpose

- Building server-side applications and network applications efficiently.

Q4: How does Node.js differ from traditional server-side programming languages?

Code:

Node.js differs from traditional server-side languages such as Java, PHP, and Python in several ways:

1. **JavaScript-Based** – Node.js uses JavaScript for server-side development, allowing developers to use the same language for both front-end and back-end development.
2. **Event-Driven Architecture** – It follows an event-driven model, where actions are triggered by events, making it efficient for handling multiple requests.
3. **Non-Blocking I/O** – Node.js performs input/output operations asynchronously, so it does not wait for one task to finish before starting another.
4. **Single-Threaded** – Unlike many traditional server technologies that create multiple threads for handling requests, Node.js uses a single-threaded event loop to manage concurrent connections efficiently.
5. **High Scalability** – Because of its asynchronous nature, Node.js can handle a large number of simultaneous connections with lower resource consumption.
6. **Performance** – Node.js uses Google's V8 JavaScript engine, which compiles JavaScript into machine code, resulting in fast execution.

Node.js	Traditional Server-side Languages
Uses JavaScript.	Often use Java, PHP, Python, C#, etc.
Event-driven and non-blocking.	Often thread-based and blocking.
Handles many concurrent connections efficiently.	May require multiple threads/processes.
Lightweight and fast for I/O operations.	Can consume more resources.

Q5: Name some real-world applications or websites built with Node.js and explain how they benefit from it.

Ans:

Many popular companies use Node.js because of its speed, scalability, and ability to handle large numbers of users efficiently.

1. Netflix

- Uses Node.js to improve streaming performance and reduce startup time.
- Handles millions of user requests efficiently.

2. PayPal

- Adopted Node.js to speed up development and improve application performance.
- Enables faster processing of web requests.

3. LinkedIn

- Uses Node.js for its mobile backend.
- Improves scalability and reduces server resource usage.

4. Uber

- Uses Node.js for real-time communication between drivers and passengers.
- Efficiently handles thousands of simultaneous connections.

5. Walmart

- Uses Node.js to manage high traffic during shopping events.
- Provides better performance and scalability.

Q6: List out the limitations of Node.js.

Ans:

Although Node.js is powerful and widely used, it has some limitations:

1. Not Suitable for CPU-Intensive Tasks

- Heavy computations such as image processing, video encoding, or complex calculations can block the event loop and reduce performance.

2. Single-Threaded Architecture

- Node.js uses a single thread for handling requests. If one operation takes too long, it can affect the responsiveness of the application.

3. Callback Complexity

- Managing multiple asynchronous operations can lead to "callback hell," making code difficult to read and maintain.

4. Less Effective for Complex Multi-Threaded Applications

- Applications requiring extensive parallel processing may perform better with technologies designed for multi-threading.

5. Rapidly Changing Ecosystem

- Frequent updates to packages and frameworks can sometimes create compatibility and maintenance issues.

6. Error Handling in Asynchronous Code

- Debugging and handling errors in asynchronous operations can be more challenging than in synchronous programming.

Q7: Outline the steps to download and install Node.js on a local machine.

Ans:

Follow these steps to download and install Node.js:

1. Open a web browser and visit the official Node.js website: [Node.js Official Website](https://nodejs.org/)
2. Download the **LTS (Long Term Support)** version, which is recommended for most users.
3. Run the downloaded installer file.
4. Accept the license agreement and click **Next**.
5. Choose the installation location (or keep the default location).
6. Select the required components and continue with the default settings.
7. Click **Install** and wait for the installation to complete.
8. Click **Finish** after the installation is successful.

Q8: Define NPM and explain its role in the Node.js ecosystem.

Ans:

NPM (Node Package Manager) is the default package manager for Node.js. It is installed automatically when Node.js is installed and is used to manage JavaScript packages and project dependencies.

Roles of NPM:

1. **Package Installation** – Allows developers to install libraries and frameworks required for a project.
2. **Dependency Management** – Keeps track of all project dependencies through the package.json file.
3. **Package Updates** – Helps update and maintain packages to their latest versions.
4. **Script Execution** – Enables running custom scripts such as testing, building, and starting applications.
5. **Package Sharing** – Allows developers to publish and share their own packages with the community.

Example:

```
npm install express
```

The above command installs the Express framework for use in a Node.js project.

Q9: How can you confirm that Node.js and NPM are installed and working correctly?

Ans:

After installing Node.js, you can verify the installation by opening **Command Prompt** (Windows) or **Terminal** (Linux/macOS) and running the following commands:

Check Node.js version:

```
node -v
```

Example output:

```
v22.0.0
```

Check NPM version:

```
npm -v
```

Example output:

```
10.5.0
```

If both commands display version numbers without errors, it confirms that **Node.js and NPM are installed and functioning correctly** on your system.

Q10: Create a basic Node.js script that prints a message to the console and explain how to execute it.

Code:

```
console.log("Hii My Name is Vatsal!!");
```

Output:

```
PS C:\Users\BAPS\Desktop\Vatsal> node .\main.js
Hii My Name is Vatsal!!
PS C:\Users\BAPS\Desktop\Vatsal> █
```